

补种未成活胡杨

题目描述

近些年来，我国防沙治沙取得显著成果。

某沙漠新种植 N 棵胡杨（编号1-N），排成一排。一个月后，有 M 棵胡杨未能成活。

现可补种胡杨 K 棵，请问如何补种（只能补种，不能新种），可以得到最多的连续胡杨树？

输入描述

N 总种植数量

- $1 \leq N \leq 100000$

M 未成活胡杨数量，M 个空格分隔的数，按编号从小到大排列

- $1 \leq M \leq N$

K 最多可以补种的数量，

- $0 \leq K \leq M$

输出描述

最多的连续胡杨棵树

用例

输入	5
	2

	2 4 1
输出	3
说明	补种到2或4结果一样，最多的连续胡杨棵树都是3。

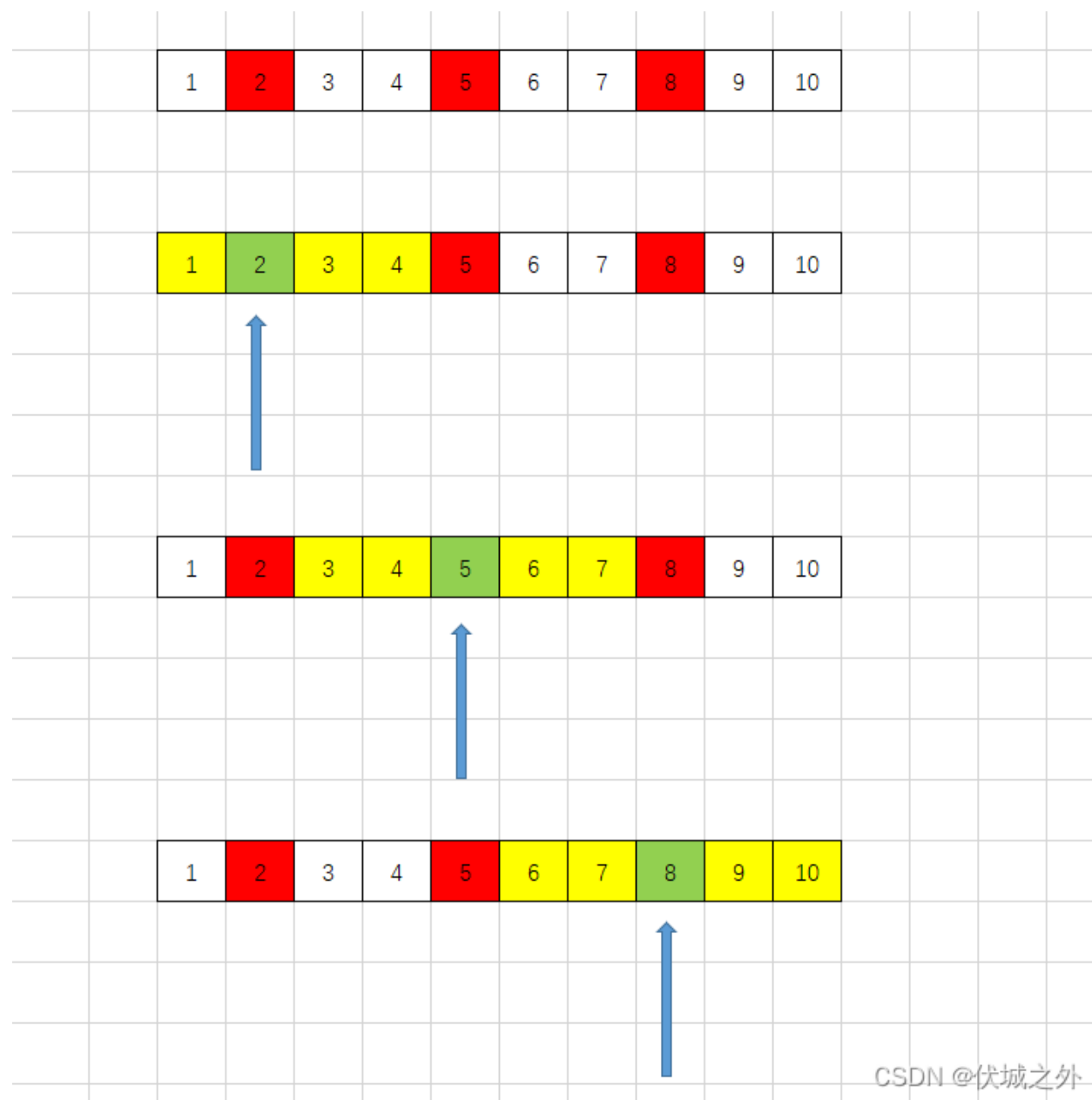
输入	10 3 2 4 7 1
输出	6
说明	补种第7棵树，最多连续胡杨树棵数位6（5，6，7，8，9，10）

题目解析

如下图中，红色 数字表示 死树的编号。

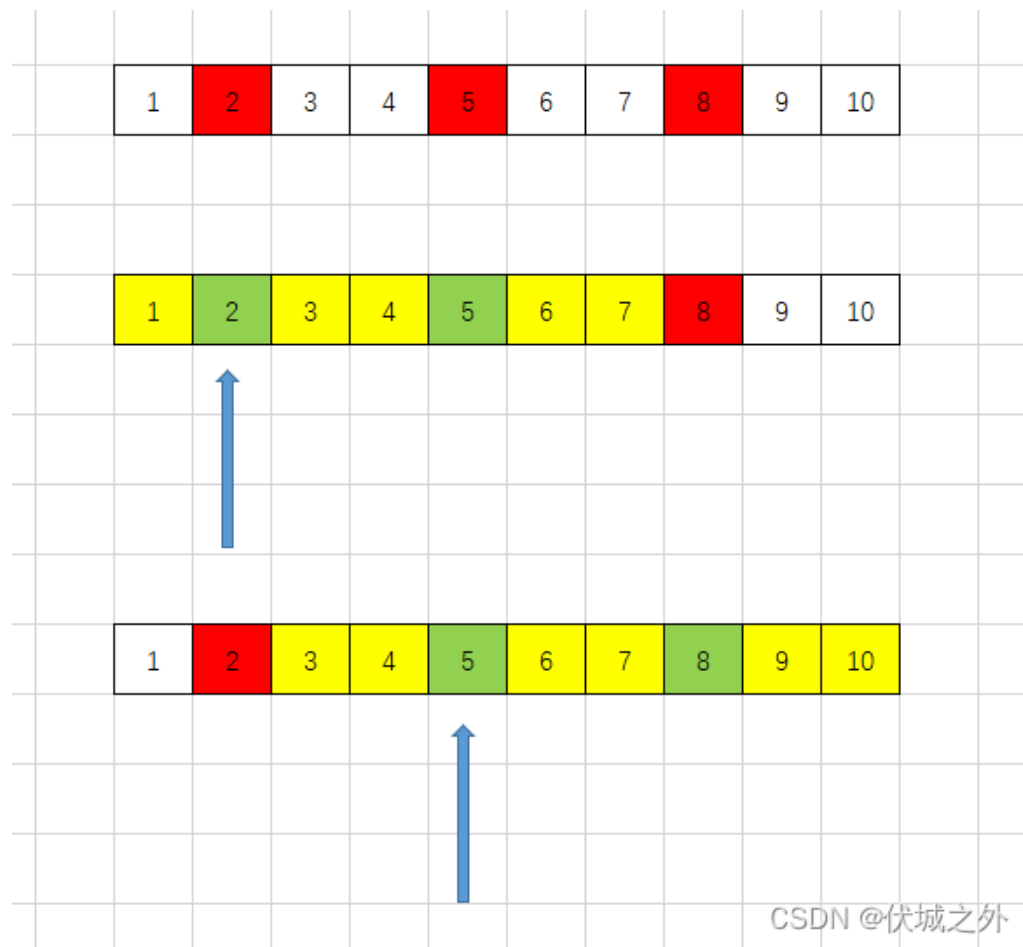
现在我们能够补种 K 棵树，即填充 K 个红色死树格子。

比如 k == 1 时，有如下填充方案，黄色+绿色部分就是连续的活的胡杨树区域。



CSDN @伏城之外

比如 $k == 2$ ，则填充方案如下：



CSDN @伏城之外

总树有 n 棵，死树有 m 棵，且死树编号保存在 `dead` 数组中，补种 k 棵。

比如 `dead = [2, 5, 8]`， $k = 2$

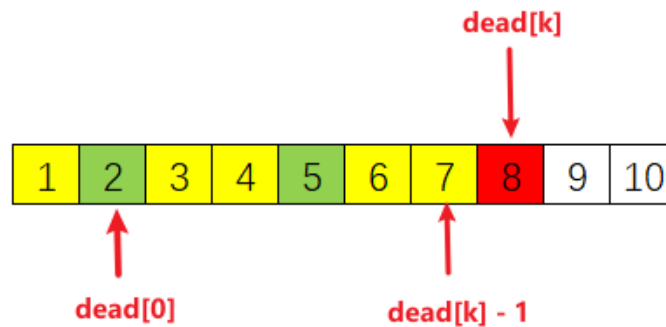
- 如果从 `dead[0]` 开始补种的话，则可以补种 `dead[0]` 和 `dead[1]`，用完 k 棵树。
- 如果从 `dead[1]` 开始补种的话，则可以补种 `dead[1]` 和 `dead[2]`，用完 k 棵树。
- 如果从 `dead[2]` 开始补种的话，则可以只能补种 `dead[2]`，用不完 k 棵树。

因此 `dead` 数组补种起始位置范围是 $[0, m - k]$

下面

- 从 `dead[0]` 开始补种（向左可以包含开头编号1的胡杨树），此时连续胡杨树的

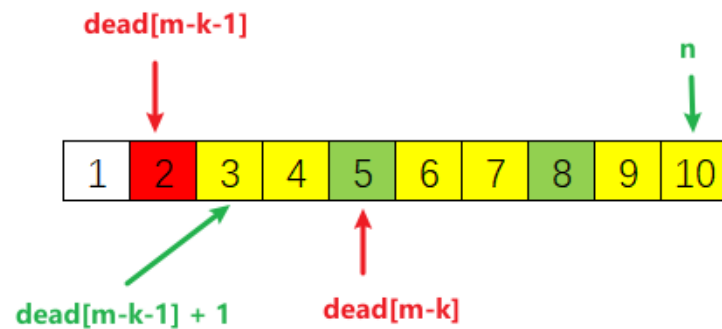
- 开始编号：1
- 结束编号： $\text{dead}[0+k] - 1$
- 长度： $(\text{dead}[0+k] - 1) - 1 + 1$



CSDN @程序员阿甘

- 从 `dead[m-k]` 开始补种（向右可以包含结尾编号为 n 的胡杨树），此时连续胡杨树的

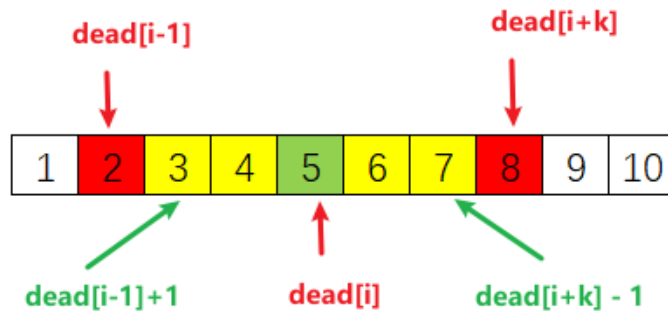
- 开始编号： $\text{dead}[m-k-1] + 1$
- 结束编号： n
- 长度： $n - (\text{dead}[m-k-1] + 1) + 1$



CSDN @程序员阿甘

- 如果从其余 $dead[i]$ 开始补种，此时连续胡杨树的：

1. 开始编号： $dead[i-1] + 1$
2. 结束编号： $dead[i+k] - 1$
3. 长度： $(dead[i+k] - 1) - (dead[i-1] + 1) + 1$



CSDN @程序员阿甘

2023.09.09

上面逻辑应该考虑下 $m == k$ 的情况，即补种数 == 未成活数，此时可以直接返回 n ，即补种后所有树都成活了。

JS算法源码

运行

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const n = parseInt(await readline());
7      const m = parseInt(await readline());
8      const dead = (await readline()).split(" ").map(Number);
9      const k = parseInt(await readline());
10
11      console.log(slide(n, m, dead, k));
12  })();
13
14  /**
15   * 滑动窗口
16   * @param {number} n 总共多少棵树
17   * @param {number} m 总共死了多少棵树
18   * @param {Array} dead 死树序号，从1开始
19   * @param {number} k 补种多少棵树
20   * @return {number} 连续最长活树的长度
21   */
22  function slide(n, m, dead, k) {
23      if (m == k) {
24          return n;
25      }
26
27      let maxLen = 0;
28      for (let i = 0; i <= m - k; i++) {
29          if (i === 0) {
30              maxLen = Math.max(maxLen, dead[i + k] - 1);
31          } else if (i === m - k) {
32              maxLen = Math.max(maxLen, n - dead[i - 1]);
33          } else {
34              maxLen = Math.max(maxLen, dead[i + k] - dead[i - 1] - 1);
35          }
36      }
37  }
```

```
36 | }
    | 37 |
38 | return maxlen;
39 | }
```

,

Java算法源码

```
1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          int n = sc.nextInt();
8          int m = sc.nextInt();
9
10         int[] dead = new int[m];
11         for (int i = 0; i < m; i++) {
12             dead[i] = sc.nextInt();
13         }
14
15         int k = sc.nextInt();
16
17         System.out.println(getResult(n, m, dead, k));
18     }
19
20     /**
21      * @param n 总共多少棵树
22      * @param m 总共死了多少棵树
23      * @param dead 数组，元素是死树序号，从1开始
24      * @param k 补种多少棵树
25      * @return 连续最长活树的长度
26      */
27     public static int getResult(int n, int m, int[] dead, int k) {
```



```

28         if (m == k) return n;
29
30     int maxLen = 0;
31
32     for (int i = 0; i <= m - k; i++) {
33         if (i == 0) {
34             maxLen = Math.max(maxLen, dead[i + k] - 1);
35         } else if (i == m - k) {
36             maxLen = Math.max(maxLen, n - dead[i - 1]);
37         } else {
38             maxLen = Math.max(maxLen, dead[i + k] - dead[i - 1] - 1);
39         }
40     }
41     return maxLen;
42 }
43 }

```

Python算法源码

```

1  # 输入获取
2  n = int(input())
3  m = int(input())
4  dead = list(map(int, input().split()))
5  k = int(input())
6
7
8  # 算法入口
9  def getResult():
10     if m == k:
11         return n
12
13     maxLen = 0
14
15     for i in range(m - k + 1):
16         if i == 0:

```

```

17         maxLen = max(maxLen, dead[i + k] - 1)
18         elif i == m - k:
19             maxLen = max(maxLen, n - dead[i - 1])
20         else:
21             maxLen = max(maxLen, dead[i + k] - dead[i - 1] - 1)
22
23     return maxLen
24
25
26 # 算法调用
27 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int getResult(int n, int m, const int dead[], int k) {
5      if (m == k) return n;
6
7      int maxLen = 0;
8
9      for (int i = 0; i <= m - k; i++) {
10         if (i == 0) {
11             maxLen = (int) fmax(maxLen, dead[i + k] - 1);
12         } else if (i == m - k) {
13             maxLen = (int) fmax(maxLen, n - dead[i - 1]);
14         } else {
15             maxLen = (int) fmax(maxLen, dead[i + k] - dead[i - 1] - 1);
16         }
17     }
18
19     return maxLen;
20 }
21

```

```

22 | int main() {
23 |     int n;
24 |     scanf("%d", &n);
25 |
26 |     int m;
27 |     scanf("%d", &m);
28 |
29 |     int dead[m];
30 |     for (int i = 0; i < m; i++) {
31 |         scanf("%d", &dead[i]);
32 |     }
33 |
34 |     int k;
35 |     scanf("%d", &k);
36 |
37 |     printf("%d\n", getResult(n, m, dead, k));
38 |
39 |     return 0;
40 | }
41 |

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int solution(int n, int m, vector<int> &dead, int k) {
6 |     if (m == k) return n;
7 |
8 |     int maxLen = 0;
9 |
10 |    for (int i = 0; i <= m - k; i++) {
11 |        if (i == 0) {
12 |            maxLen = max(maxLen, dead[i + k] - 1);

```

复制

运行

```

13         } else if (i == m - k) {14 |             maxLen = max(maxLen, n - dead[i - 1]);
15         } else {
16             maxLen = max(maxLen, dead[i + k] - dead[i - 1] - 1);
17         }
18     }
19
20     return maxLen;
21 }
22
23 int main() {
24     int n, m;
25     cin >> n >> m;
26
27     vector<int> dead(m);
28     for (int i = 0; i < m; i++) {
29         cin >> dead[i];
30     }
31
32     int k;
33     cin >> k;
34
35     cout << solution(n, m, dead, k) << endl;
36
37     return 0;
38 }

```